

VetLetters SaaS — Replit Build Instructions

Comprehensive Technical Specification for AI-Powered VA Claims Assistant

Project Overview

Build a full-stack SaaS web application that helps veterans generate professional VA claims documents using AI. Veterans pay a monthly subscription to access an intake questionnaire, AI-generated letters (nexus, personal statements, buddy letters, NODs), rating estimator, and document management dashboard.

Stack: React (Vite) + Node.js/Express + Supabase + Stripe + Anthropic Claude API

1. Tech Stack & Dependencies

Frontend

```
React 18 (Vite)
React Router v6
Tailwind CSS
shadcn/ui component library
React Hook Form + Zod validation
React PDF (for PDF preview)
Axios
Stripe.js (@stripe/stripe-js)
```

Backend

```
Node.js 20+
Express.js
Supabase JS client (@supabase/supabase-js)
Stripe Node SDK (stripe)
Anthropic SDK (@anthropic-ai/sdk)
Multer (file uploads)
PDF-lib (PDF generation)
jsonwebtoken
```

```
dotenv
cors
express-rate-limit
```

Services

- **Supabase** — Auth, PostgreSQL database, file storage
 - **Stripe** — Subscriptions and payments
 - **Anthropic Claude API** — Document generation (claude-sonnet-4-20250514)
 - **Resend or SendGrid** — Transactional email
-

2. Environment Variables

Backend `.env`

```
# Supabase
SUPABASE_URL=your_supabase_url
SUPABASE_SERVICE_KEY=your_service_role_key

# Stripe
STRIPE_SECRET_KEY=sk_live...
STRIPE_WEBHOOK_SECRET=whsec...
STRIPE_BASIC_PRICE_ID=price...
STRIPE_PRO_PRICE_ID=price...
STRIPE_CONCIERGE_PRICE_ID=price...

# Anthropic
ANTHROPIC_API_KEY=sk-ant-...

# App
PORT=3001
JWT_SECRET=your_jwt_secret
FRONTEND_URL=https://your-app.replit.app

# Email
RESEND_API_KEY=re...
FROM_EMAIL=noreply@vetletters.com
```

Frontend `.env`

```
VITE_API_URL=https://your-backend.replit.app
VITE_SUPABASE_URL=your_supabase_url
VITE_SUPABASE_ANON_KEY=your_anon_key
VITE_STRIPE_PUBLISHABLE_KEY=pk_live_...
```

3. Database Schema (Supabase / PostgreSQL)

Run these SQL migrations in Supabase SQL editor:

```
-- Users profile (extends Supabase auth.users)
CREATE TABLE profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email TEXT NOT NULL,
  full_name TEXT,
  phone TEXT,
  stripe_customer_id TEXT UNIQUE,
  subscription_tier TEXT CHECK (subscription_tier IN ('none', 'basic', 'pro', 'co
  subscription_status TEXT DEFAULT 'inactive',
  stripe_subscription_id TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Veteran profile (claim info)
CREATE TABLE veteran_profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  -- Military Service
  branch TEXT, -- Army, Navy, Air Force, Marines, Coast Guard, Space Force
  rank TEXT,
  mos_rate TEXT, -- Military Occupational Specialty
  service_start_date DATE,
  service_end_date DATE,
  discharge_type TEXT, -- Honorable, General, OTH, etc.
  deployment_locations TEXT[], -- Array of locations
  -- VA Info
  va_file_number TEXT,
  current_rating INTEGER, -- 0-100
  -- Personal
  date_of_birth DATE,
  ssn_last4 TEXT,
  address TEXT,
  city TEXT,
```

```

state TEXT,
zip TEXT,
-- Exposure
agent_orange_exposure BOOLEAN DEFAULT FALSE,
camp_lejeune BOOLEAN DEFAULT FALSE,
burn_pit_exposure BOOLEAN DEFAULT FALSE,
gulf_war_service BOOLEAN DEFAULT FALSE,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW(),
UNIQUE(user_id)
);

-- Service-connected conditions
CREATE TABLE conditions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    condition_name TEXT NOT NULL,
    icd10_code TEXT,
    current_rating INTEGER DEFAULT 0,
    claimed_rating INTEGER,
    service_connected BOOLEAN DEFAULT FALSE,
    date_of_diagnosis DATE,
    treating_physician TEXT,
    notes TEXT,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Service incidents/events (nexus connectors)
CREATE TABLE service_incidents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    condition_id UUID REFERENCES conditions(id),
    incident_date DATE,
    location TEXT,
    description TEXT NOT NULL,
    witnesses TEXT[],
    documented BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Generated documents
CREATE TABLE documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    document_type TEXT NOT NULL CHECK (document_type IN (
        'nexus_letter', 'personal_statement', 'buddy_letter',
        'nod', 'secondary_condition', 'cue_motion', 'increase_claim'
    ))

```

```

)),
title TEXT NOT NULL,
condition_id UUID REFERENCES conditions(id),
content TEXT NOT NULL, -- Generated text
pdf_path TEXT, -- Supabase storage path
status TEXT DEFAULT 'draft' CHECK (status IN ('draft', 'final', 'submitted')),
ai_model TEXT DEFAULT 'claude-sonnet-4-20250514',
generation_prompt TEXT,
word_count INTEGER,
created_at TIMESTAMPTZ DEFAULT NOW(),
updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- Uploaded files (medical records, C&P exams, etc.)
CREATE TABLE uploaded_files (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    file_name TEXT NOT NULL,
    file_type TEXT NOT NULL,
    storage_path TEXT NOT NULL,
    file_size INTEGER,
    extracted_text TEXT,
    document_category TEXT CHECK (document_category IN (
        'medical_record', 'cp_exam', 'service_record', 'buddy_statement', 'other'
    )),
    processed BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Rating estimates
CREATE TABLE rating_estimates (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    conditions JSONB NOT NULL, -- [{condition, rating}]
    combined_rating INTEGER,
    estimated_monthly_benefit INTEGER,
    breakdown JSONB,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Usage tracking (for tier limits)
CREATE TABLE usage_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    action TEXT NOT NULL, -- 'generate_document', 'upload_file', 'estimate_rating'
    metadata JSONB,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```
);

-- Enable Row Level Security
ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE veteran_profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE conditions ENABLE ROW LEVEL SECURITY;
ALTER TABLE service_incidents ENABLE ROW LEVEL SECURITY;
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;
ALTER TABLE uploaded_files ENABLE ROW LEVEL SECURITY;
ALTER TABLE rating_estimates ENABLE ROW LEVEL SECURITY;
ALTER TABLE usage_logs ENABLE ROW LEVEL SECURITY;

-- RLS Policies (users only see their own data)
CREATE POLICY "Users own data" ON profiles FOR ALL USING (auth.uid() = id);
CREATE POLICY "Users own data" ON veteran_profiles FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "Users own data" ON conditions FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "Users own data" ON service_incidents FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "Users own data" ON documents FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "Users own data" ON uploaded_files FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "Users own data" ON rating_estimates FOR ALL USING (auth.uid() = user_id);
CREATE POLICY "Users own data" ON usage_logs FOR ALL USING (auth.uid() = user_id);

-- Supabase Storage Buckets
-- Create these in Supabase dashboard:
-- Bucket: "medical-records" (private, max 50MB per file)
-- Bucket: "generated-pdfs" (private, max 10MB per file)
```

4. Project File Structure

```
/vetletters-saas
├── /frontend (React/Vite)
│   ├── /src
│   │   ├── /components
│   │   │   ├── /ui (shadcn components)
│   │   │   ├── Layout.jsx
│   │   │   ├── Navbar.jsx
│   │   │   ├── ProtectedRoute.jsx
│   │   │   ├── SubscriptionGate.jsx
│   │   │   └── DocumentCard.jsx
│   │   ├── /pages
│   │   │   ├── Landing.jsx
│   │   │   ├── Login.jsx
│   │   │   ├── Register.jsx
│   │   │   └── Dashboard.jsx
```

```
| | | └─ Intake.jsx
| | | └─ Conditions.jsx
| | | └─ GenerateDocument.jsx
| | | └─ Documents.jsx
| | | └─ RatingEstimator.jsx
| | | └─ FileUpload.jsx
| | | └─ Pricing.jsx
| | | └─ Checkout.jsx
| | | └─ Settings.jsx
| | └─ /hooks
| | | └─ useAuth.js
| | | └─ useSubscription.js
| | | └─ useDocuments.js
| | └─ /services
| | | └─ api.js
| | | └─ supabase.js
| | | └─ stripe.js
| | └─ /context
| | | └─ AuthContext.jsx
| | └─ App.jsx
| | └─ main.jsx
| └─ package.json
| └─ vite.config.js
|
└─ /backend (Node/Express)
  └─ /routes
    └─ auth.js
    └─ profile.js
    └─ conditions.js
    └─ documents.js
    └─ generate.js
    └─ files.js
    └─ rating.js
    └─ stripe.js
    └─ webhooks.js
  └─ /middleware
    └─ auth.js (JWT verification)
    └─ subscription.js (tier checking)
    └─ rateLimit.js
  └─ /services
    └─ anthropic.js
    └─ supabase.js
    └─ stripe.js
    └─ pdfGenerator.js
    └─ email.js
  └─ /prompts
    └─ nexusLetter.js
```

```
| | | └─ personalStatement.js
| | | └─ buddyLetter.js
| | | └─ nod.js
| | | └─ secondaryCondition.js
| | └─ server.js
| └─ package.json
```

5. Backend Implementation

server.js

```
const express = require('express');
const cors = require('cors');
const rateLimit = require('express-rate-limit');
require('dotenv').config();

const app = express();

// Stripe webhook needs raw body
app.use('/api/webhooks/stripe', express.raw({ type: 'application/json' }));
app.use(express.json({ limit: '10mb' }));
app.use(cors({ origin: process.env.FRONTEND_URL, credentials: true }));

// Rate limiting
const limiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 100 });
const aiLimiter = rateLimit({ windowMs: 60 * 60 * 1000, max: 20 }); // 20 AI call
app.use('/api/', limiter);
app.use('/api/generate', aiLimiter);

// Routes
app.use('/api/auth', require('./routes/auth'));
app.use('/api/profile', require('./routes/profile'));
app.use('/api/conditions', require('./routes/conditions'));
app.use('/api/documents', require('./routes/documents'));
app.use('/api/generate', require('./routes/generate'));
app.use('/api/files', require('./routes/files'));
app.use('/api/rating', require('./routes/rating'));
app.use('/api/stripe', require('./routes/stripe'));
app.use('/api/webhooks', require('./routes/webhooks'));

app.listen(process.env.PORT || 3001, () => {
  console.log(`Server running on port ${process.env.PORT || 3001}`);
});
```

middleware/auth.js

```
const { createClient } = require('@supabase/supabase-js');

const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SERVICE_KEY
);

module.exports = async (req, res, next) => {
  const token = req.headers.authorization?.replace('Bearer ', '');
  if (!token) return res.status(401).json({ error: 'Unauthorized' });

  const { data: { user }, error } = await supabase.auth.getUser(token);
  if (error || !user) return res.status(401).json({ error: 'Invalid token' });

  req.user = user;
  next();
};
```

middleware/subscription.js

```
const { supabase } = require('../services/supabase');

const TIER_LEVELS = { none: 0, basic: 1, pro: 2, concierge: 3 };

module.exports = (requiredTier) => async (req, res, next) => {
  const { data: profile } = await supabase
    .from('profiles')
    .select('subscription_tier, subscription_status')
    .eq('id', req.user.id)
    .single();

  if (!profile || profile.subscription_status !== 'active') {
    return res.status(403).json({ error: 'Active subscription required', code: 'N'
  }

  if (TIER_LEVELS[profile.subscription_tier] < TIER_LEVELS[requiredTier]) {
    return res.status(403).json({
      error: `${requiredTier} tier required`,
      code: 'UPGRADE_REQUIRED',
      currentTier: profile.subscription_tier
    });
  }
}
```

```
    req.subscription = profile;
    next();
};
```

services/anthropic.js

```
const Anthropic = require('@anthropic-ai/sdk');

const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

async function generateDocument(systemPrompt, userPrompt, maxTokens = 2000) {
    const response = await client.messages.create({
        model: 'claude-sonnet-4-20250514',
        max_tokens: maxTokens,
        system: systemPrompt,
        messages: [{ role: 'user', content: userPrompt }]
    });
    return response.content[0].text;
}

module.exports = { generateDocument };
```

routes/generate.js

```
const router = require('express').Router();
const auth = require('../middleware/auth');
const checkSub = require('../middleware/subscription');
const { generateDocument } = require('../services/anthropic');
const { supabase } = require('../services/supabase');
const { generatePDF } = require('../services/pdfGenerator');
const prompts = require('../prompts');

// Generate a document
router.post('/', auth, checkSub('basic'), async (req, res) => {
    try {
        const { document_type, condition_id, additional_context } = req.body;
        const userId = req.user.id;

        // Fetch veteran profile + condition
        const { data: vetProfile } = await supabase
            .from('veteran_profiles')
            .select('*')
            .eq('user_id', userId)
            .single();
```

```

const { data: condition } = condition_id ? await supabase
  .from('conditions')
  .select('*', service_incidents('*'))
  .eq('id', condition_id)
  .single() : { data: null };

// Fetch any uploaded file extracts (Pro+ only)
let medicalContext = '';
if (req.subscription.subscription_tier !== 'basic') {
  const { data: files } = await supabase
    .from('uploaded_files')
    .select('extracted_text, document_category')
    .eq('user_id', userId)
    .eq('processed', true)
    .not('extracted_text', 'is', null);

  if (files?.length) {
    medicalContext = files.map(f => `${f.document_category}: ${f.extracted_
  }
}

// Build prompts
const promptBuilder = prompts[document_type];
if (!promptBuilder) return res.status(400).json({ error: 'Invalid document ty

const { system, user } = promptBuilder({ vetProfile, condition, medicalContex

// Generate with Claude
const content = await generateDocument(system, user, 2500);

// Save to database
const { data: doc } = await supabase.from('documents').insert({
  user_id: userId,
  document_type,
  title: `${document_type.replace(/_/g, ' ').replace(/\b\w/g, l => l.toUpperCase
  condition_id,
  content,
  status: 'draft',
  word_count: content.split(' ').length
}).select().single();

// Generate PDF
const pdfPath = await generatePDF(doc.id, content, doc.title);
await supabase.from('documents').update({ pdf_path: pdfPath }).eq('id', doc.i

// Log usage

```

```

    await supabase.from('usage_logs').insert({
      user_id: userId,
      action: 'generate_document',
      metadata: { document_type, condition_id }
    });

    res.json({ document: { ...doc, pdf_path: pdfPath }, content });
  } catch (err) {
    console.error('Generate error:', err);
    res.status(500).json({ error: 'Generation failed', details: err.message });
  }
});

module.exports = router;

```

routes/stripe.js

```

const router = require('express').Router();
const Stripe = require('stripe');
const stripe = new Stripe(process.env.STRIPE_SECRET_KEY);
const auth = require('../middleware/auth');
const { supabase } = require('../services/supabase');

const PRICE_IDS = {
  basic: process.env.STRIPE_BASIC_PRICE_ID,
  pro: process.env.STRIPE_PRO_PRICE_ID,
  concierge: process.env.STRIPE_CONCIERGE_PRICE_ID
};

// Create checkout session
router.post('/create-checkout', auth, async (req, res) => {
  const { tier } = req.body;
  if (!PRICE_IDS[tier]) return res.status(400).json({ error: 'Invalid tier' });

  const { data: profile } = await supabase
    .from('profiles')
    .select('stripe_customer_id, email')
    .eq('id', req.user.id)
    .single();

  let customerId = profile?.stripe_customer_id;
  if (!customerId) {
    const customer = await stripe.customers.create({
      email: profile.email,
      metadata: { supabase_id: req.user.id }
    });
  }
});

```

```

    customerId = customer.id;
    await supabase.from('profiles').update({ stripe_customer_id: customerId }).eq
  }

const session = await stripe.checkout.sessions.create({
  customer: customerId,
  mode: 'subscription',
  payment_method_types: ['card'],
  line_items: [{ price: PRICE_IDS[tier], quantity: 1 }],
  success_url: `${process.env.FRONTEND_URL}/dashboard?success=true`,
  cancel_url: `${process.env.FRONTEND_URL}/pricing`,
  metadata: { supabase_id: req.user.id, tier }
});

res.json({ url: session.url });
});

// Customer portal
router.post('/portal', auth, async (req, res) => {
  const { data: profile } = await supabase
    .from('profiles')
    .select('stripe_customer_id')
    .eq('id', req.user.id)
    .single();

  const session = await stripe.billingPortal.sessions.create({
    customer: profile.stripe_customer_id,
    return_url: `${process.env.FRONTEND_URL}/settings`
  });

  res.json({ url: session.url });
});

module.exports = router;

```

routes/webhooks.js

```

const router = require('express').Router();
const Stripe = require('stripe');
const stripe = new Stripe(process.env.STRIPE_SECRET_KEY);
const { supabase } = require('../services/supabase');

router.post('/stripe', async (req, res) => {
  const sig = req.headers['stripe-signature'];
  let event;

```

```

try {
  event = stripe.webhooks.constructEvent(req.body, sig, process.env.STRIPE_WEBH
} catch (err) {
  return res.status(400).send(`Webhook Error: ${err.message}`);
}

const getSupabaseId = async (customerId) => {
  const customer = await stripe.customers.retrieve(customerId);
  return customer.metadata.supabase_id;
};

const TIER_MAP = {
  [process.env.STRIPE_BASIC_PRICE_ID]: 'basic',
  [process.env.STRIPE_PRO_PRICE_ID]: 'pro',
  [process.env.STRIPE_CONCIERGE_PRICE_ID]: 'concierge'
};

switch (event.type) {
  case 'customer.subscription.created':
  case 'customer.subscription.updated': {
    const sub = event.data.object;
    const userId = await getSupabaseId(sub.customer);
    const tier = TIER_MAP[sub.items.data[0]?.price.id] || 'basic';

    await supabase.from('profiles').update({
      subscription_tier: tier,
      subscription_status: sub.status === 'active' ? 'active' : 'inactive',
      stripe_subscription_id: sub.id
    }).eq('id', userId);
    break;
  }
  case 'customer.subscription.deleted': {
    const sub = event.data.object;
    const userId = await getSupabaseId(sub.customer);
    await supabase.from('profiles').update({
      subscription_tier: 'none',
      subscription_status: 'inactive',
      stripe_subscription_id: null
    }).eq('id', userId);
    break;
  }
}

res.json({ received: true });
});

module.exports = router;

```

6. AI Prompt Templates

`prompts/nexusLetter.js`

```
module.exports = function nexusLetterPrompt({ vetProfile, condition, medicalConte
  const system = `You are an expert VA claims writer with 20+ years of experience
service connection. You write professional, medical-grade nexus letters that clea
between a veteran's military service and their current medical condition.
```

Your letters:

- Use precise medical terminology
- Reference relevant VA regulations (38 CFR, M21-1)
- Cite the "at least as likely as not" (50%+) standard
- Are structured for maximum persuasiveness
- Are written in first person from the veteran's perspective
- Never make false claims – only support what the evidence shows
- Include a clear nexus statement

Format: Professional letter with date placeholder, salutation, body paragraphs, a

```
  const user = `Generate a nexus letter for this veteran:
```

MILITARY SERVICE:

- Branch: \${vetProfile?.branch || 'Not specified'}
- Service Dates: \${vetProfile?.service_start_date} to \${vetProfile?.service_end_d
- MOS/Rate: \${vetProfile?.mos_rate || 'Not specified'}
- Deployments: \${vetProfile?.deployment_locations?.join(', ') || 'Not specified'}
- Exposures: \${[
 vetProfile?.agent_orange_exposure && 'Agent Orange',
 vetProfile?.camp_lejeune && 'Camp Lejeune contaminated water',
 vetProfile?.burn_pit_exposure && 'Burn pit/airborne hazards',
 vetProfile?.gulf_war_service && 'Gulf War service'
].filter(Boolean).join(', ') || 'None documented'}

CLAIMED CONDITION: \${condition?.condition_name || 'Not specified'}

Diagnosis Date: \${condition?.date_of_diagnosis || 'Not specified'}

Current Rating: \${condition?.current_rating || 0}%

Seeking Rating: \${condition?.claimed_rating || 'Not specified'}%

SERVICE INCIDENTS CONNECTING TO CONDITION:

```
${condition?.service_incidents?.map(i =>
```

```
  `- ${i.incident_date || 'Unknown date'} at ${i.location || 'Unknown location'}:
).join('\n') || 'Veteran to provide details'}
```

```

${medicalContext ? `MEDICAL RECORDS SUMMARY:\n${medicalContext}\n` : ''}
${additional_context ? `ADDITIONAL CONTEXT:\n${additional_context}` : ''}

Write a complete, professional nexus letter establishing service connection.`;

    return { system, user };
};

```

prompts/index.js

```

module.exports = {
  nexus_letter: require('./nexusLetter'),
  personal_statement: require('./personalStatement'),
  buddy_letter: require('./buddyLetter'),
  nod: require('./nod'),
  secondary_condition: require('./secondaryCondition'),
  increase_claim: require('./increaseClaim')
};

```

Create similar prompt files for:

- `personalStatement.js` — First-person account of how condition affects daily life
- `buddyLetter.js` — Template for fellow veteran/witness statements
- `nod.js` — Notice of Disagreement for denied/low-rated claims
- `secondaryCondition.js` — Letter connecting secondary condition to primary
- `increaseClaim.js` — Letter supporting increased rating based on worsening

7. Rating Estimator Logic

routes/rating.js

```

const router = require('express').Router();
const auth = require('../middleware/auth');

// VA "whole person" combined rating calculator
function calculateCombinedRating(ratings) {
  if (!ratings.length) return 0;

```

```

const sorted = [...ratings].sort((a, b) => b - a);
let remaining = 100;

for (const rating of sorted) {
  const disability = (rating / 100) * remaining;
  remaining -= disability;
}

const combined = Math.round(100 - remaining);

// VA rounds to nearest 10 (with 5+ rounding up)
return Math.round(combined / 10) * 10;
}

// Monthly benefit lookup (2024 rates - update annually)
const MONTHLY_RATES = {
  0: 0, 10: 175, 20: 346, 30: 537, 40: 774,
  50: 1102, 60: 1395, 70: 1759, 80: 2044,
  90: 2297, 100: 3737
};

router.post('/estimate', auth, async (req, res) => {
  const { conditions } = req.body; // [{name, rating}]

  if (!conditions?.length) {
    return res.status(400).json({ error: 'Conditions required' });
  }

  const ratings = conditions.map(c => c.rating).filter(r => r > 0);
  const combined = calculateCombinedRating(ratings);
  const monthly = MONTHLY_RATES[combined] || 0;

  // What-if analysis: what rating needed to reach next tier?
  const nextTier = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100].find(r => r > combin

  res.json({
    combined_rating: combined,
    estimated_monthly: monthly,
    individual_ratings: conditions,
    next_tier: nextTier,
    next_tier_monthly: nextTier ? MONTHLY_RATES[nextTier] : null,
    monthly_increase_potential: nextTier ? (MONTHLY_RATES[nextTier] - monthly) :
    // Special flags
    tdiu_eligible: combined >= 60 && ratings.some(r => r >= 60), // One condition
    smc_eligible: combined === 100,
    pact_act_note: 'If you served post-9/11, check PACT Act presumptives at benef
  });

```

```
});
```

```
module.exports = router;
```

8. Frontend Pages

Routing (App.jsx)

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider } from '../context/AuthContext';
import ProtectedRoute from '../components/ProtectedRoute';

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          { /* Public */ }
          <Route path="/" element={<Landing />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />
          <Route path="/pricing" element={<Pricing />} />

          { /* Protected - any authenticated user */ }
          <Route element={<ProtectedRoute />}>
            <Route path="/dashboard" element={<Dashboard />} />
            <Route path="/intake" element={<Intake />} />
            <Route path="/conditions" element={<Conditions />} />
            <Route path="/rating" element={<RatingEstimator />} />
            <Route path="/checkout" element={<Checkout />} />
            <Route path="/settings" element={<Settings />} />
          </Route>

          { /* Protected - subscription required */ }
          <Route element={<ProtectedRoute requireSubscription />}>
            <Route path="/generate" element={<GenerateDocument />} />
            <Route path="/documents" element={<Documents />} />
            <Route path="/upload" element={<FileUpload />} />
          </Route>
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  );
};
```

}

Intake Form (Multi-Step)

Build a multi-step wizard with these steps:

1. **Military Service** — Branch, dates, MOS, discharge type
2. **Deployments & Exposures** — Locations, Agent Orange, burn pits, Camp Lejeune, PACT Act
3. **Current Conditions** — List conditions with current/claimed ratings
4. **Service Incidents** — Link incidents to conditions (what happened, when, where)
5. **VA Info** — Current rating, file number, prior claims
6. **Review & Save**

Use React Hook Form + Zod for validation. Save progress to Supabase on each step completion.

Generate Document Page

1. Select document type (dropdown with icons)
2. Select condition to write about
3. Optional: additional context textarea
4. Preview button — shows AI output in modal
5. Download PDF button
6. Save to Documents button

Dashboard

Show:

- Current subscription tier + quick upgrade CTA
 - Combined rating (calculated from saved conditions)
 - Documents generated this month
 - Recent documents list
 - Quick action buttons: Generate Letter, Add Condition, Upload Records
 - Tip of the day (PACT Act, C&P exam prep, etc.)
-

9. Subscription Tiers & Feature Gates

Feature	Basic \$19	Pro \$49	Concierge \$149
Nexus letters	✓ 5/mo	✓ Unlimited	✓ Unlimited
Personal statements	✓	✓	✓
Buddy letter templates	✓	✓	✓
NOD drafts	✗	✓	✓
Secondary condition letters	✗	✓	✓
Medical record upload + AI analysis	✗	✓ (up to 10 files)	✓ Unlimited
Rating estimator	✓	✓	✓
PDF downloads	✓	✓	✓
Human review (VSO/attorney)	✗	✗	✓
Priority support	✗	✗	✓

Enforce limits using `usage_logs` table counts per billing period.

10. Security Requirements

- All routes except `/auth/*`, `/`, `/pricing` require JWT auth middleware
- Document generation routes require active subscription check
- File uploads: validate MIME type server-side (only PDF, JPEG, PNG, DOCX)
- File uploads: scan for malware using ClamAV or cloud service before storing
- Rate limit AI generation endpoints (20 requests/hour per user)
- Never expose `SUPABASE_SERVICE_KEY` to frontend
- Store all files in private Supabase buckets with signed URLs (1-hour expiry)
- Use Stripe's webhook signature verification — never trust client-side subscription status
- Input sanitization on all AI prompt fields to prevent injection

- Log all document generation for audit trail
-

11. Email Notifications (Resend)

Send emails for:

- Welcome email on registration
 - Subscription confirmed
 - Document generated (with PDF attached or link)
 - Subscription expiring in 7 days
 - Monthly usage summary
-

12. Legal/Compliance Notices

Add these to the app:

1. **Disclaimer on all generated documents:**

"This document was generated with AI assistance and is intended as a draft template. VetLetters is not a law firm and does not provide legal advice. Review all documents before submission and consider consulting an accredited VA claims agent or attorney."

2. **Privacy Policy** — How veteran/medical data is stored and protected
 3. **Terms of Service** — No guarantees of claim outcomes
 4. **HIPAA notice** — Medical records are encrypted at rest and in transit
-

13. Deployment (Replit)

Backend Replit config

- Language: Node.js
- Run command: `node server.js`
- Set all env vars in Replit Secrets tab

- Enable "Always On" (paid Replit plan)

Frontend Replit config

- Language: Node.js (Vite)
- Build command: `npm run build`
- Run command: `npm run preview` or `serve dist/ folder`
- Set `VITE_API_URL` to backend Replit URL

Stripe Webhook

After deploying backend, register webhook endpoint in Stripe Dashboard:

```
https://your-backend.replit.app/api/webhooks/stripe  
Events: customer.subscription.created, customer.subscription.updated, customer.su
```

14. MVP Launch Checklist

- Auth (register, login, logout, password reset)
- Veteran profile intake (multi-step form)
- Add conditions + service incidents
- Nexus letter generation (Claude API)
- PDF download
- Stripe checkout (Basic tier only for MVP)
- Stripe webhook updates subscription status
- Dashboard with recent documents
- Rate estimator
- Mobile responsive
- Disclaimer on all documents
- Privacy policy page
- Terms of service page

- Error handling + loading states everywhere
- Test Stripe in test mode before going live

Ship with just Basic tier. Add Pro/Concierge after first 10 paying customers.

15. Growth / Phase 2 Features

- PACT Act presumptive condition checker (auto-flag based on service dates/locations)
- C&P exam preparation guide (AI-generated based on their conditions)
- Claim status tracker (manual entry with timeline view)
- Buddy letter request system (send email to fellow veteran to fill out template)
- Accredited VSO referral marketplace (Concierge tier)
- Mobile app (React Native)
- Multi-veteran household accounts
- VA form pre-fill (21-526EZ, 21-4142 release forms)